



← Back to student guides

GitHub Actions

CI/CD

3 levels

139 sections



# The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

EVENT

WORKFLOW

JOB

STEP

**24**

sections

**26**

examples





Mid-level and senior take the same topics further, nothing here is thrown away.

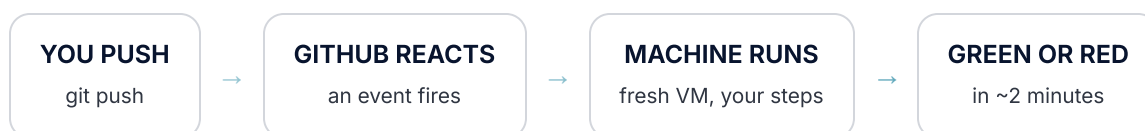
Each section ends with a **Try it** task. Do them as you go. They take a few minutes each, and the concepts stick only once you have watched your own run go green.

## What is GitHub Actions?

GitHub Actions is GitHub's **built-in automation platform**. You describe work you want done in a text file, commit that file to your repository, and GitHub runs the work for you on a computer it creates on demand.

Compare that to what it replaces.

Before this existed, automating a project meant running a separate machine (a Jenkins server, a TeamCity box, something in a cupboard) keeping it patched, giving it credentials to reach your code, and hoping the person who configured it three years ago left notes. Actions removes that entire layer. There is no server to maintain, because GitHub provisions a fresh machine for each piece of work and destroys it afterwards.



Two consequences of that design explain most of what follows. Notice them now rather than discovering them later.

**The instructions live inside your repository.** That means your automation is versioned, branched, and reviewed exactly like your source code. If a pipeline breaks, `git log` tells you who changed it and why. You can propose a change to it in a pull request and have a colleague review it before it takes effect. This sounds administrative; in practice it is the single biggest reason teams stop being afraid of their build system.

**Every run starts on a clean machine.** There is no leftover state from last time, no "it works on the build server because someone installed something there in 2019". This is a feature, it



What people use it for:



### Run tests automatically

Every push and pull request gets built and tested without anyone remembering to do it.



### Deploy on merge

Code that passed its checks ships itself, to a server, a registry, or a package index.



### Scheduled jobs

Nightly builds, weekly dependency checks, database backups, cleanup tasks.



### Repository chores

Label issues, greet new contributors, close stale threads, publish release notes.

To follow along you need a free GitHub account, the ability to make a commit, and a repository. Any repository, in any language, and a brand-new empty one is the best place to experiment. Public repositories get unlimited free minutes, so a playground costs you nothing.

#### ► TRY IT

1. Create a public repository called `actions-playground` . It can be empty.
2. Open its **Actions** tab. GitHub will offer you starter workflows. Do not pick one yet, just look.
3. Open any popular open-source repository you use and click its **Actions** tab.

**Expected result** – your own tab is empty, and the popular project's tab shows dozens of runs with green ticks and red crosses. That list is what you are about to start producing.

## What are CI and CD?



## The problem they solve

Imagine three people working on the same project. Every time someone finishes a change, somebody has to remember to run the tests, check the formatting, build the application, and put it on a server. Nobody enjoys that, so it gets skipped when people are busy. Broken code reaches the main branch on a Friday and nobody notices until Monday. Someone deploys from their laptop, and their laptop happens to have a different library version than the server, so it works locally and breaks in production.

Every one of those failures has the same root cause: **a human being is doing the repetitive work**. Humans are excellent at judgement and terrible at performing the same twelve steps identically four hundred times.

## Continuous Integration

**CI** means every change is automatically built and tested the moment it appears.

The word "integration" is about merging work together. Historically, developers would work in isolation for weeks and then merge, a painful event because everyone's changes conflicted at once. Continuous integration means merging small changes frequently and verifying each one immediately. If someone breaks the project, they find out in four minutes rather than four days, while the change is still fresh in their mind and small enough to understand.

In practice, CI is a machine that on every push: fetches your code, installs dependencies, runs the linter, runs the type checker, runs the tests, and reports pass or fail.

## Continuous Delivery and Continuous Deployment

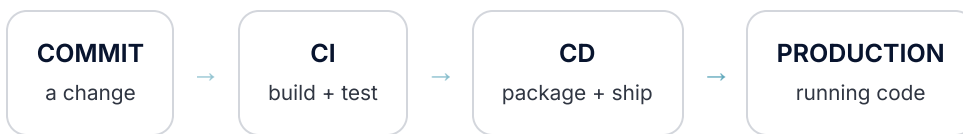
**CD** means that once a change has passed all of its checks, shipping it is also automatic.

Confusingly, CD stands for two related things:



|                              |                                                                                 |                          |
|------------------------------|---------------------------------------------------------------------------------|--------------------------|
| Continuous <b>Delivery</b>   | Every passing change is automatically packaged and made <b>ready</b> to release | A person clicks "deploy" |
| Continuous <b>Deployment</b> | Every passing change is automatically released to production                    | None                     |

Most teams practise continuous delivery and describe it as continuous deployment. The distinction matters in interviews, and it matters when you design a pipeline: adding a human approval gate turns one into the other, and Actions supports both.



## What this looks like in practice

Put together, CI/CD is a machine that watches your repository and does the boring, critical work identically every single time. The benefits compound in ways that are hard to appreciate until you have worked without them:

| Without CI/CD                            | With CI/CD                                     |
|------------------------------------------|------------------------------------------------|
| "Did you run the tests?"                 | The tests ran, and the report is on the commit |
| Broken <b>main</b> discovered days later | Broken <b>main</b> discovered in minutes       |
| Deploys are a scary manual ritual        | Deploys are a merge                            |
| "It works on my machine"                 | It works on a clean machine, every time        |
| Release notes written from memory        | Release notes generated from commits           |
| Only one person knows how to ship        | The pipeline is the documentation              |

### Where Actions fits

GitHub Actions is a **CI/CD platform**: a way to implement both practices. Jenkins, GitLab CI, CircleCI, and Travis do the same job. What distinguishes Actions is that it lives inside



► TRY IT

1. Pick a project you work on. Write down every manual step between "I finished the code" and "it is running for users".
2. Mark each step **CI** (proving the change is safe) or **CD** (shipping it).
3. Mark the ones that are identical every time. Those are the ones a machine should do.

**Expected result** – a list of five to ten steps, most of them mechanical. That list is the pipeline you are going to build by the end of this page.

## Where workflow files live, and why the path is fixed

Instructions for Actions go in files called **workflows**, and the location is exact. Get it wrong and GitHub ignores the file: no error, nothing in the Actions tab, no hint that anything is missing.

your repository

Copy

```
my-project/
├── .github/
│   ├── workflows/           ← must be exactly this
│   │   ├── ci.yml           ← becomes active on commit
│   │   └── release.yml      ← so does this
├── src/
└── README.md
```

Any file in that folder ending `.yml` or `.yaml` becomes an active workflow the moment it is committed. The leading dot on `.github` is intentional: on Linux and macOS it makes the folder hidden, which is why you may not see it in a file browser.

### Three ways people get this wrong



message.

#### ▶ TRY IT

1. In your playground repository, click **Add file** → **Create new file**.
2. Type the filename as `.github/workflows/notes.yml` and note how GitHub turns each slash into a folder as you type.
3. Put a single comment in the file: `# scratch space`. Commit it.

**Expected result** – the folders now exist in your repository, and the Actions tab still shows nothing, because a workflow with no trigger can never run. You have created the container, not the content.

## YAML in five minutes, for workflow authors

Workflow files are written in **YAML**, a text format designed to be read by people rather than to be efficient for machines. Almost everything you will write is covered by six rules.

| Rule                   | Looks like                                  | Means                                           |
|------------------------|---------------------------------------------|-------------------------------------------------|
| Key and value          | <code>name: CI</code>                       | Setting <code>name</code> is <code>CI</code>    |
| Two-space indent nests | <code>jobs:</code> then <code>build:</code> | <code>build</code> belongs to <code>jobs</code> |
| Dash makes a list item | <code>- run: npm test</code>                | One item in a list                              |
| Hash is a comment      | <code># runs on PRs</code>                  | Ignored by the machine                          |
| Quote version numbers  | <code>'3.11'</code> not <code>3.11</code>   | Unquoted becomes the number 3.1                 |
| <code>`</code>         | <code>`</code> keeps a multi-line block     | <code>`run:</code>                              |



item at the same level in the same column.

YAML

Copy

```
name: Example          # a key with a text value

on: push               # another key/value pair

jobs:                  # a key whose value is a nested structure
  build:               #   two more spaces, so build belongs to jobs
    runs-on: ubuntu-latest
    steps:              #   a key whose value is a list
      - run: echo one   #     list item
      - run: |           #     list item with a multi-line value
        echo two
        echo three
```

### The tab character will waste an hour of your life

A single tab used for indentation is a hard parse error, and the error message never mentions tabs. Configure your editor to insert spaces in YAML files before you write anything real. There is a [.editorconfig](#) for exactly this in the Tips & Tricks section.

#### ► TRY IT

1. Open the `notes.yml` you created and write the `Example` block above into it, by hand rather than pasting.
2. Deliberately break it: remove two spaces from in front of `runs-on` . Commit.
3. Check the Actions tab, then fix the indentation and commit again.

**Expected result** – the broken version appears in the Actions tab as a failed run with a YAML error, proof that GitHub reads the file. Reading that error message once now saves you confusion later.



Enough theory. Create `.github/workflows/hello.yml`. You can do this entirely in the browser via **Add file** → **Create new file**.

`.github/workflows/hello.yml`

Copy

```
name: My First Workflow      # label shown in the Actions tab

on: push                    # the trigger: run on every push

jobs:
  greet:                    # job id – you choose this name
    runs-on: ubuntu-latest  # ask GitHub for a Linux machine
    steps:
      - name: Say hello
        run: echo "Hello from a machine that did not exist a minute ago"

      - name: Show run info
        run: |
          echo "OS           : ${ runner.os }"
          echo "Repo        : ${ github.repository }"
          echo "Pushed by   : ${ github.actor }"
```

Commit it. Committing `is itself a push`, so the workflow triggers immediately. Open the **Actions** tab and watch it run.

Now let me walk through every line, because this small file contains the entire conceptual model.

`name: My First Workflow` is a human label. Omit it and GitHub shows the file path instead, which is why unnamed workflows make the Actions tab hard to read.

`on: push` is the **trigger**, the doorbell. This says "run whenever commits reach any branch". Without an `on:` key a workflow can never run at all, which is why your `notes.yml` did nothing.

`jobs:` opens the list of work. `greet:` is a **job id** that you invent; it identifies this job in the interface and in any dependency you declare later.



`steps:` is the ordered list of things to do on that machine. Each `- name: / run:` pair is one step: `name` is the label in the log, `run` is a shell command executed on the runner.

The `|` in the second step means "the following indented block is one multi-line string", so those three lines run as a small shell script within a single step.

`${{ ... }}` is an **expression**: GitHub substitutes a real value there before the command runs. `runner.os` becomes `Linux`; `github.repository` becomes `your-name/your-repo`.

## What happened while you were watching



0s

### Event fires

Your commit created a `push` event describing the branch, the commit, and who pushed it.



1s

### Workflow matched

GitHub read every file in `.github/workflows/` and found one whose `on:` matched.



5s

### Machine created

A clean Ubuntu virtual machine booted. Your code was **not** on it, so remember this.



8s

### Steps ran in order

Expressions were replaced with real values, then each command executed top to bottom.



14s

### Machine destroyed

Everything on it is gone. The next push repeats the recipe on a brand-new machine.

#### ► TRY IT

1. Commit the workflow above, then open the run and expand both steps to read their output.
2. Add a third step `- run: date` and find its output in the next run.
3. Change `on: push` to `on: workflow_dispatch`, commit, then use the **Run workflow** button in the Actions tab.
4. Break it on purpose with `- run: exit 1`. Watch the red cross appear, open the log, then fix it.



the single most useful thing on this page. Reading a failure log early is the skill everything else depends on.

## The six building blocks, nested

Every workflow you will ever read is built from six nested concepts. Getting this hierarchy solid is worth more than memorising a hundred YAML keys.

### EVENTS. ANYTHING THAT HAPPENS IN THE REPOSITORY

#### push

A branch or tag moved

#### pull\_request

Opened, synchronised, reopened

#### schedule • workflow\_dispatch

Cron, or a button



### WORKFLOW: ONE YAML FILE IN .GITHUB/WORKFLOWS/

#### Declares its triggers, then its jobs

One file per pipeline. Several files can watch the same event



### JOBS: PARALLEL BY DEFAULT, EACH ON ITS OWN RUNNER



Starts at the same time as `test`

`deploy`

`needs: [test, lint]` , the only way to order



### Steps

`run` a command, or `uses` an action

### Runner

The machine: GitHub-hosted or self-hosted

### Artifacts · cache

The only way files cross a job boundary

**The line that matters:** steps share a machine, jobs do not. Everything else in this guide follows from that one fact: `runs-on` belongs to a job, parallelism is free, and handing files between jobs needs artifacts.

An **event** is something that happened in your repository. A **workflow** is a file saying which events it cares about and what to do. A **job** is a named group of work running on one machine. A **step** is a single task inside a job. A **runner** is the machine itself. An **action** is a reusable step someone has already written and published.

The relationship that trips up almost everyone is between jobs and steps, so let me be explicit:

#### ✓ Steps share everything

- Same machine, same filesystem
- Run in the order written
- Files one step creates, the next can read
- Installed tools persist between them
- A failure stops the rest of the job

#### ✗ Jobs share nothing

- Each gets its own fresh machine
- Run `at the same time` by default
- Files do not cross between them
- No shared installed tools
- Ordered only by `needs`



consequences rather than tidiness. Two jobs run in parallel and finish sooner, but they cannot hand files to each other without extra machinery.

### Start with one job and several steps

Reach for multiple jobs when you want parallelism or a hard ordering gate. Splitting too early creates the "my deploy job cannot find the build output" problem for no benefit.

#### ► TRY IT

1. Add a step that creates a file: `- run: echo "hello" > note.txt`.
2. Add another step after it: `- run: cat note.txt`. Commit and confirm it works.
3. Now move the `cat` step into a

**Expected result** – second job  
with its own `runs-on`, and run it again.

**Expected result** – the two-step version prints "hello". The two-job version fails with "No such file or directory". You have just proved to yourself that jobs share no filesystem. This one experiment prevents a whole category of future confusion.

## Triggers: what starts a workflow

`on:` decides when a workflow runs, and it is where a lot of a pipeline's usefulness comes from. These are the triggers worth knowing on day one.

| Trigger                   | Fires when                      | Use it for                        |
|---------------------------|---------------------------------|-----------------------------------|
| <code>push</code>         | Commits reach a branch or tag   | Test every change                 |
| <code>pull_request</code> | A PR opens, updates, or reopens | Check <code>before</code> merging |



| <code>workflow_dispatch</code>                   | You click <b>Run Workflow</b>      | Manual runs, deploys       |
|--------------------------------------------------|------------------------------------|----------------------------|
| <code>schedule</code>                            | A cron time matches ( <b>UTC</b> ) | Nightly jobs, cleanups     |
| <code>release</code>                             | You publish a release              | Build and attach downloads |
| <code>issues</code> , <code>issue_comment</code> | Issue activity                     | Triage bots                |

You can list several at once, and you can narrow each one so it only fires in the situations you care about:

YAML

Copy

```
on:
  push:
    branches: [main]           # only main, not every branch
    tags: ['v*']               # and any v-prefixed tag
    paths-ignore: ['docs/**', '**/*.md'] # but not doc-only changes
  pull_request:
    branches: [main]          # PRs targeting main
    types: [opened, synchronize, reopened]
  schedule:
    - cron: '0 3 * * *'       # 03:00 UTC daily
  workflow_dispatch:          # and a manual button
```

The five cron fields are `minute hour day-of-month month day-of-week`, so `'0 3 * * *'` reads as "minute 0 of hour 3, any day, any month, any weekday". Two things surprise people: schedules are always **UTC**, never your local time, and they only run from your repository's default branch, so testing one on a feature branch will drive you mad.

## Manual runs with inputs

`workflow_dispatch` can ask for values, which turns a workflow into a small internal tool:

YAML

Copy



```
inputs:
  environment:
    description: Where to deploy
    type: choice
    options: [staging, production]
    default: staging
  dry-run:
    description: Print actions without doing them
    type: boolean
    default: true

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - run: |
          echo "target : ${ inputs.environment }"
          echo "dry run: ${ inputs.dry-run }"
```

Inputs support `string` , `boolean` , `choice` , and `environment` types, and you read them through the `inputs` context.

### The most common reason a workflow "does not run"

Read `on:` before reading anything else. It is nearly always a `branches` filter you forgot, a file in the wrong folder, or a `schedule` you are waiting for in the wrong timezone. When something does not run, the trigger is the suspect, not the steps.

#### ► TRY IT

1. Change your workflow's trigger to `branches: [main]` , then push a commit to a new branch called `test-filter` .
2. Confirm nothing runs. Then open a pull request from that branch and add `pull_request:` to the trigger list.
3. Add the `workflow_dispatch` block with the `environment` choice input, and run it manually twice, once for each option.

**Expected result** – the branch push does nothing (correct behaviour that looks like a bug), the pull request triggers a run, and the manual runs print the value you



## Jobs: parallel by default, ordered with `needs`

Jobs run at the same time unless you say otherwise. `needs` creates ordering.

YAML

Copy

```
jobs:
  lint:                                # lint and test start together
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: ruff check .

  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pytest -q

  deploy:
    needs: [lint, test]                # waits for BOTH to succeed
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy.sh
```

`needs` builds a dependency graph. A job with no `needs` starts immediately; a job with `needs` waits for every listed job to succeed. If any of them fails, the dependent job is **skipped** rather than failed, a distinction that matters when you read a run summary.

The practical effect is that pipeline shape is a design choice. Three independent checks as three jobs finish in the time of the slowest one; the same three as steps in one job take the sum of all three. But the parallel version needs artifacts to move files between stages, and the sequential version does not.



1. write a workflow with three jobs ( `a` , `b` , and `c` ) each running `sleep 20` and echoing its name.
2. Run it and note the total wall-clock time in the run summary.
3. Now add `needs: a` to `b` and `needs: b` to `c` . Run again and compare.

**Expected result** – the first version takes about 20 seconds in total; the second takes about 60. The run graph at the top of the page draws the difference, so you can now see what `needs` costs you.

## Steps: `run` a command versus `uses` an action

Every step is either a shell command or a prebuilt component. That is the entire taxonomy.

|               | <code>run</code>                    | <code>uses</code>                        |
|---------------|-------------------------------------|------------------------------------------|
| Is            | A shell command on the runner       | A published, reusable action             |
| Written       | <code>- run: pytest -q</code>       | <code>- uses: actions/checkout@v4</code> |
| Configured by | Arguments                           | A <code>with:</code> block               |
| For           | What you already type in a terminal | Setup, caching, uploads, cloud logins    |

A single step is one or the other. Putting `run` and `uses` in the same step is an error, and a favourite interview trick question.

YAML

Copy

```
steps:
  - uses: actions/checkout@v4          # action, no configuration

  - uses: actions/setup-node@v4        # action, configured
    with:
      node-version: '20'
```



```
- name: Test with options          # multi-line, custom shell, working dir
  working-directory: ./backend
  shell: bash
  env:
    CI: 'true'
  run: |
    npm run lint
    npm test -- --coverage
```

Those step options solve real problems, so learn them now. `working-directory` runs the command somewhere other than the repository root. `shell` picks the interpreter: `bash` , `sh` , `pwsh` , `python` . `env` sets variables for that step alone. `name` is what appears in the log, which is the difference between a readable run and a wall of truncated commands.

### The number-one beginner failure

**The runner starts empty.** Your repository is not on it. `actions/checkout` is the action that clones it. If you see "no such file or directory" for a file you can plainly see on GitHub, this is why, and it is why almost every workflow's first step is that line.

#### ► TRY IT

1. Write a workflow whose only step is `- run: ls -la` . Commit and read the output.
2. Now add `- uses: actions/checkout@v4` as the step before it, and run again.
3. Add a step with `working-directory: .github` running `ls -la` .

**Expected result** – the first run lists an almost-empty directory. The second lists your actual repository. The third lists the contents of `.github` only. You have now seen, rather than read, why `checkout` exists.

## Actions and the Marketplace



### actions/checkout

Clones your repository onto the runner. First step of nearly every job.



### actions/setup-\*

Installs a language version. `setup-node` , `setup-python` , `setup-java` , `setup-go` .



### actions/cache

Reuses downloaded dependencies between runs, so later runs are much faster.



### actions/upload-artifact

Saves files off the machine before it is destroyed, so you can download them.



### actions/download-artifact

Pulls those files into a later job.



### actions/github-script

Runs JavaScript against the GitHub API: comment on a PR, add a label.

An action is referenced as `owner/repo@ref` , and the `ref` decides what code you execute:

| Reference                                | Meaning                               | When                         |
|------------------------------------------|---------------------------------------|------------------------------|
| <code>actions/checkout@v4</code>         | Major-version tag, moves with patches | Default for official actions |
| <code>actions/checkout@v4.1.7</code>     | One exact release                     | You want reproducibility     |
| <code>actions/checkout@8f4b7f8...</code> | An exact commit                       | Third-party actions          |
| <code>actions/checkout@main</code>       | A branch                              | Almost never                 |

**Never point at a branch on someone else's action**



SHA if the action is not from a name you recognise. Senior covers the full supply-chain picture.

#### ► TRY IT

1. Open the Marketplace and find an action that posts a comment on a pull request.
2. Read its README: note the `with:` inputs it accepts and the `permissions` it asks for.
3. Click through to its repository and find the commit SHA of its latest release.

**Expected result** – you can now read any action's documentation and know what to write. Finding the SHA is the habit that becomes mandatory at Senior level.

## Runners: the machine each job gets

`runs-on` picks the machine. Three labels cover nearly everything: `ubuntu-latest`, `windows-latest`, and `macos-latest`. Ubuntu is the fastest and cheapest, and the default choice unless you have a specific reason otherwise.

GitHub's Linux runners come with a generous set of preinstalled tools: Git, Docker, several versions of Python and Node, common build toolchains, and the `gh` CLI. You often need less setup than you expect.

|             | GitHub-hosted               | Self-hosted                   |
|-------------|-----------------------------|-------------------------------|
| Who runs it | GitHub                      | You                           |
| Lifetime    | New per job, then destroyed | Persistent                    |
| Cost        | Free on public repos        | Your hardware                 |
| Pick it     | Almost always               | GPU, licence, private network |



a paid licence, or access to a private network. Senior covers when that is justified and the risks it introduces.

#### ► TRY IT

1. Add a step that prints versions: `python3 --version; node --version; docker --version; git --version`.
2. Run it on `ubuntu-latest`.
3. Change `runs-on` to `windows-latest` and run again. You will need `shell: bash` for the same commands to work.

**Expected result** – Ubuntu already has everything, which is why you rarely install Docker or Git yourself. The Windows run also shows you why `shell:` exists.

## Expressions and contexts: reading the run

Expressions live inside `${{ }}` and read from **contexts**, read-only objects describing the current run. This is how a workflow becomes aware of its own circumstances.

| Context              | Gives you                                | Examples                                                                                                                                           |
|----------------------|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>github</code>  | Event and repo data                      | <code>github.ref</code> , <code>github.sha</code> , <code>github.actor</code> ,<br><code>github.event_name</code> , <code>github.repository</code> |
| <code>runner</code>  | The machine                              | <code>runner.os</code> , <code>runner.temp</code>                                                                                                  |
| <code>env</code>     | Variables you set                        | <code>env.LOG_LEVEL</code>                                                                                                                         |
| <code>secrets</code> | Encrypted secrets                        | <code>secrets.GITHUB_TOKEN</code>                                                                                                                  |
| <code>inputs</code>  | <code>workflow_dispatch</code><br>inputs | <code>inputs.environment</code>                                                                                                                    |
| <code>steps</code>   | Earlier step outputs                     | <code>steps.build.outputs.tag</code>                                                                                                               |



|        |                         |                             |
|--------|-------------------------|-----------------------------|
| needs  | Earlier job outputs     | needs.build.outputs.version |
| matrix | The current combination | matrix.python               |

YAML

Copy

```
- name: Print useful context values
  run: |
    echo "ref      : ${github.ref}"           # refs/heads/main
    echo "sha      : ${github.sha}"
    echo "event    : ${github.event_name}"    # push / pull_request
    echo "actor    : ${github.actor}"
    echo "run number : ${github.run_number}"
    echo "workspace : ${github.workspace}"
```

These are substituted **before** the command runs, which changes how you read them: by the time your shell sees the line, `${github.ref}` has already become literal text. That is why they work in any part of a workflow file, not just inside `run`.

#### ► TRY IT

1. Add the printing step above to your workflow and run it from a branch.
2. Note what `github.ref` prints. Write it down.
3. Now open a pull request and run it again. Compare `github.ref` and `github.event_name`.

**Expected result** — `refs/heads/your-branch` on a push, but `refs/pull/N/merge` on a pull request. Knowing that `github.ref` is the **full** ref, and that it differs by event, prevents the single most common condition bug.

## Conditions with `if`



YAML

Copy

```
jobs:
  deploy:
    if: github.ref == 'refs/heads/main'      # job-level condition
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Only on pull requests
        if: github.event_name == 'pull_request'
        run: ./pr-comment.sh

      - name: Only for version tags
        if: startsWith(github.ref, 'refs/tags/v')
        run: ./publish.sh

      - name: Run even if an earlier step failed
        if: always()
        run: ./collect-logs.sh

      - name: Only when something failed
        if: failure()
        run: ./notify-team.sh
```

| You want                  | Write                                                  |
|---------------------------|--------------------------------------------------------|
| Only on <code>main</code> | <code>if: github.ref == 'refs/heads/main'</code>       |
| Only on pull requests     | <code>if: github.event_name == 'pull_request'</code>   |
| Only on a version tag     | <code>if: startsWith(github.ref, 'refs/tags/v')</code> |
| Skip a bot                | <code>if: github.actor != 'dependabot[bot]'</code>     |
| Even after a failure      | <code>if: always()</code>                              |
| Only after a failure      | <code>if: failure()</code>                             |



stops the ones after it.

### The gotcha that costs an hour

`github.ref` is the **full** ref: `refs/heads/main` , not `main` . Comparing it to `'main'` silently never matches, and nothing in the log explains why. When a condition misbehaves, print the value first: `- run: echo "${{ github.ref }}"` .

#### ► TRY IT

1. Add a step with the

**Expected result** – wrong

condition on purpose: `if: github.ref == 'main'` . Push to main and watch it skip.

2. Fix it to `refs/heads/main` and watch it run.

3. Add a step with `if: failure()` and a step above it that runs `exit 1` .

**Expected result** – the skipped step appears greyed out with a "skipped" marker rather than an error, which is why this bug is hard to spot. The `failure()` step runs only in the red run, which is the pattern you will use for notifications.

## Environment variables, and their three scopes

Variables have three scopes, and the most specific one wins:

YAML

Copy

```
env:
  APP_NAME: checkout-api      # every job, every step

jobs:
  test:
    runs-on: ubuntu-latest
    env:
```



```
env:  
  LOG_LEVEL: trace      # this step only – wins
```

The complication is that **each run step is a new shell process**, so `export` does not survive to the next step. GitHub solves this with special files whose paths are provided as environment variables. Appending a line to `$GITHUB_ENV` sets a variable for every **later** step in the same job:

YAML

Copy

```
- name: Compute a version and share it  
  run: echo "VERSION=1.4.${{ github.run_number }}" >> "$GITHUB_ENV"  
  
- name: Use it in a later step  
  run: echo "building $VERSION"
```

There is also `$GITHUB_STEP_SUMMARY`, which accepts Markdown and renders it on the run page. It is the cheapest reporting you will ever add, and it means nobody has to download a file to see three numbers:

YAML

Copy

```
- name: Publish a summary  
  run: |  
    {  
      echo "### Test results"  
      echo "| metric | value |"  
      echo "|---|---|"  
      echo "| tests | 412 |"  
      echo "| coverage | 91% |"  
    } >> "$GITHUB_STEP_SUMMARY"
```

**Two limits worth remembering**



come later on this page.

#### ► TRY IT

1. Write a step that does `export F00=bar` and a second step that echoes `$F00` . Confirm it is empty.
2. Change the first step to `echo "F00=bar" >> "$GITHUB_ENV"` and run again.
3. Now try echoing `$F00` in the

**Expected result** – same

step that wrote it, and note that it is still empty.

4. Add the step-summary block and look at the bottom of the run page.

**Expected result** – the export version prints nothing, the `$GITHUB_ENV` version prints `bar` from the next step onwards but not in the writing step, and your run page now has a rendered table on it. Those three behaviours explain most "my variable is empty" confusion.

## Secrets, and what they cannot protect

Secrets are encrypted values you set in **Settings** → **Secrets and variables** → **Actions**. You read them with `${{ secrets.NAME }}` , and once stored you cannot read them back in the interface, only overwrite them.

YAML

Copy

```
- name: Deploy
  env:
    API_TOKEN: ${{ secrets.API_TOKEN }}      # pass as data
  run: ./deploy.sh                          # script reads "$API_TOKEN"
```



YAML

Copy

```
permissions:
  contents: read
  pull-requests: write

steps:
  - uses: actions/github-script@v7
    with:
      script: |
        github.rest.issues.createComment({
          issue_number: context.issue.number,
          owner: context.repo.owner,
          repo: context.repo.repo,
          body: 'Build passed ✅'
        })
```

That `permissions:` block matters now even though Senior covers it properly: it controls what the automatic token is allowed to do, and declaring it narrows the token rather than widening it.

### Three rules, from day one

**Never echo a secret.** GitHub masks known values in logs, but that is a safety net, not a strategy. It cannot mask a value you transformed.

**Pass secrets through `env`, not into the command line.** Interpolating one directly into a shell makes it far easier to leak through an error message or `set -x`.

**Fork pull requests get no secrets**, by design. That is not a bug to work around.

#### ► TRY IT

1. Add a repository secret called `TEST_SECRET` with the value `hunter2`.
2. Write a step that checks it is set

**Expected result** – without printing it

:

```
if [ -z "$TEST_SECRET" ]; then echo "missing"; exit 1; fi; echo "present" , passing it via
```



**Expected result** – the check prints "present". The echo prints `***`, so masking worked. Then try `echo "$TEST_SECRET" | base64` and watch the masking fail, which is why "never print it" is the actual rule.

## Caching: making the second run fast

The runner is new every time, which means your dependencies download every time. On a real project that is often most of the run. Caching fixes it, and for mainstream languages it is a single line:

YAML

Copy

```
- uses: actions/setup-python@v5
  with:
    python-version: '3.11'
    cache: pip                # ← the whole thing
```

YAML

Copy

```
- uses: actions/setup-node@v4
  with:
    node-version: '20'
    cache: npm                # same idea; also 'yarn', 'pnpm'
```



### run 1 Cache miss

Dependencies download normally, then get saved at the end of the job.



### run 2 Cache hit

Dependencies are restored from the cache instead of downloaded.



The mechanism underneath is a **key**: a string that identifies the cached content. The setup actions build one from your lockfile, so it changes exactly when your dependencies change. For anything the setup actions do not know about, you write the key yourself:

YAML

Copy

```
- uses: actions/cache@v4
  with:
    path: ~/.cache/my-tool
    key: ${ runner.os }-mytool-${ hashFiles('tool.lock') }
    restore-keys: |
      ${ runner.os }-mytool-
```

`hashFiles` produces a stable hash of the matched files. `restore-keys` is a fallback prefix tried when the exact key misses, giving a partial hit that is still much better than nothing. Mid-level goes considerably deeper on key design, because getting it wrong is the difference between a cache that helps and one that silently does nothing.

#### ► TRY IT

1. Add a real dependency install to your workflow: a `requirements.txt` with a few packages, or a `package.json`.
2. Run it once

**Expected result** – without `cache:` and note the install step's duration.

3. Add `cache: pip` (or `npm`), then run twice. Compare the install duration on the second run.

**Expected result** – the first cached run is no faster, it has nothing to restore, but the second is faster, and the step log says "Cache restored from key...". One line, permanently.



The machine is destroyed when the job ends, so anything you want to keep must be uploaded before that happens.

YAML

Copy

```
- name: Run tests
  run: pytest -q --junitxml=reports/junit.xml --cov --cov-report=html

- name: Keep the reports
  if: always()          # ← save them even when tests FAIL
  uses: actions/upload-artifact@v4
  with:
    name: test-reports
    path: |
      reports/
      htmlcov/
    retention-days: 7
```

Artifacts appear as downloadable files at the bottom of the run page. The `if: always()` is the important part: without it, a failing test stops the job before the upload runs, so you lose the report exactly when you need it most.

Cache and artifacts are easy to confuse, so keep this distinction:

✓ **Artifact = data you need**

- Test reports, build output, logs
- You download it, or a later job does
- If it is missing, something is broken
- Named and retained for a set period

✗ **Cache = a shortcut**

- Downloaded dependencies
- Reused by the next run
- If it is missing, the run is just slower
- Keyed and restored automatically

One sentence for interviews: a cache is an optimisation you must be able to lose; an artifact is data you cannot.

► TRY IT



3. Now add `run: exit 1`

**Expected result** – before  
the upload step and run again.

4. Add `if: always()` to the upload and run once more.

**Expected result** – run two produces no artifact at all, because the failure  
stopped the job first. Run three produces the artifact despite the red cross.  
That is the whole reason `if: always()` exists.

## Passing data between jobs, deliberately

Jobs are separate machines, so nothing crosses automatically. Strings travel as **outputs**; files travel as **artifacts**.

YAML

Copy

```
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      version: ${ steps.meta.outputs.version } # declare the output
    steps:
      - uses: actions/checkout@v4

      - id: meta # the step needs an id
        run: echo "version=1.4.${ github.run_number }" >> "$GITHUB_OUTPUT"

      - run: make build # produces dist/

      - uses: actions/upload-artifact@v4
        with:
          name: dist
          path: dist/

  deploy:
    needs: build
```



```
with:
  name: dist
  path: dist/

- run: ./deploy.sh --version "${{ needs.build.outputs.version }}"
```

Three things have to line up for a job output to work, and missing any one produces an empty value with no error: the step needs an `id`, it must write to `$GITHUB_OUTPUT`, and the **job** must declare `outputs`: mapping a name to that step's output. Then the dependent job reads it via the `needs` context.

#### ► TRY IT

1. Build the two-job example above, but have `build` just create a text file rather than a real build.
2. Confirm `deploy` prints the version and can read the downloaded file.
3. Now remove the `outputs`: block from the `build` job and run again.

**Expected result** – the version becomes empty: no error, just a blank. That silent failure is why the three-part requirement is worth memorising.

## Matrix: testing several versions at once

Copying a job four times to test four language versions is the wrong answer. A **matrix** generates them for you:

YAML

Copy

```
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false          # let every version report
    matrix:
```



```
- uses: actions/setup-python@v5
  with:
    python-version: ${{ matrix.python }}
    cache: pip
- run: pip install -r requirements.txt
- run: pytest -q
```

That produces three parallel jobs, each with `matrix.python` set to a different value. Add a second axis and they multiply:

YAML

Copy

```
matrix:
  os: [ubuntu-latest, windows-latest]
  python: ['3.11', '3.12']          # 2 x 2 = four jobs
runs-on: ${{ matrix.os }}
```

Understand `fail-fast` now. It defaults to `true`, which cancels every sibling job as soon as one fails. That is right when you want a fast red signal and wrong when you are diagnosing, because a single failing cell cancelling the rest hides the pattern you need to see.

#### ► TRY IT

1. Set up the three-Python matrix and run it. Note how the job names in the Actions tab include the version.
2. Make the test fail on one version only: `- run: if [ "${{ matrix.python }}" = "3.10" ]; then exit 1; fi .`
3. Run it with the default `fail-fast`, then with `fail-fast: false`.

**Expected result** – with fail-fast on, the other two jobs are cancelled mid-run. With it off, all three report. You now know which one you want while debugging.

## Guards every workflow should have from day one



YAML

Copy

```
jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 15          # default ceiling is SIX HOURS
    steps:
      - uses: actions/checkout@v4

      - name: Optional check
        continue-on-error: true  # a failure here does not fail the job
        run: ./experimental-lint.sh
```

| Guard                                | Why                                                          |
|--------------------------------------|--------------------------------------------------------------|
| <code>timeout-minutes</code>         | A hung job otherwise burns up to six hours of your allowance |
| <code>continue-on-error</code>       | Lets a non-critical step fail without failing the build      |
| <code>if: always()</code> on uploads | Diagnostics survive a red run                                |
| A <code>paths-ignore</code> filter   | Documentation changes do not run the full suite              |

The timeout is the one people skip and later regret. The default is six hours per job, and a workflow waiting on something that will never arrive consumes your entire monthly allowance before anyone notices.

#### ► TRY IT

1. Add `timeout-minutes: 1` to a job and give it a step that runs `sleep 120`.
2. Watch what the run looks like when the timeout fires.
3. Add a step with `continue-on-error: true` running `exit 1`, followed by a normal step.

**Expected result** – the timed-out job is marked failed with a clear "cancelled after 1 minute" message rather than hanging. The tolerated failure shows a warning marker while the job still succeeds, so note that the job is green even though a step failed.



Debugging is a skill, and it has an order.

**01 Click the red job, then the red step**

GitHub expands the failure for you automatically.

**02 Scroll up, not down**

Logs end with a tool's summary. The real cause is usually well above it.

**03 Run the command on your own machine**

If it fails there too, the workflow is innocent and you are debugging your project.

**04 Print what you are unsure about**

`pwd` , `ls -la` , `echo "$VAR"` , `python --version` . A step is just a shell.

**05 Use "Re-run failed jobs"**

Retries only the red job instead of the whole pipeline.

**06 Re-run with debug logging**

The

**Re-run**

menu has a checkbox for far more detail.

Four traps account for most beginner failures, and all four have the same root cause: a new machine each job, a new shell each step:

| Symptom                                       | Cause             | Fix                                  |
|-----------------------------------------------|-------------------|--------------------------------------|
| "File not found" for a file in your repo      | No checkout       | Add <code>actions/checkout@v4</code> |
| The deploy job cannot find <code>dist/</code> | Different machine | Upload/download an artifact          |



|                                                       |                                        |                                                 |
|-------------------------------------------------------|----------------------------------------|-------------------------------------------------|
| <code>cd</code> in one step has no effect on the next | new shell per step                     | <code>working-directory:</code> on the step     |
| A variable is empty in the next step                  | <code>export</code> dies with the step | <code>echo "K=v" &gt;&gt; "\$GITHUB_ENV"</code> |

YAML

Copy

```
# Does not work
- run: export VERSION=1.2.3
- run: echo "$VERSION"          # empty

# Works
- run: echo "VERSION=1.2.3" >> "$GITHUB_ENV"
- run: echo "$VERSION"          # 1.2.3

# Does not work
- run: cd frontend
- run: npm ci

# Works
- run: npm ci
  working-directory: frontend
```

#### ► TRY IT

1. Reproduce all four traps in one workflow deliberately, one job at a time.
2. For each, read the actual error message before fixing it.
3. Then re-run one failed job only, and re-run it again with debug logging enabled.

**Expected result** – four distinct, recognisable error messages. Having seen each one on purpose, you will recognise them instantly when they happen for real, which is the difference between a five-minute fix and an afternoon.



Everything above, in one file. Nothing here is new. Read it as a whole and you should recognise every line.

.github/workflows/ci.yml

Copy

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    paths-ignore: ['docs/**', '**/*.md']
  workflow_dispatch:

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    strategy:
      fail-fast: false
      matrix:
        python: ['3.11', '3.12']
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python }
          cache: pip

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Lint
        run: ruff check .

      - name: Test
        run: pytest -q --junitxml=reports/junit-${ matrix.python }.xml
```



```
      uses: actions/upload-artifact@v4
    with:
      name: reports-${{ matrix.python }}
      path: reports/
      retention-days: 7

build:
  needs: test
  if: github.ref == 'refs/heads/main'
  runs-on: ubuntu-latest
  timeout-minutes: 10
  outputs:
    version: ${ steps.meta.outputs.version }
  steps:
    - uses: actions/checkout@v4
    - id: meta
      run: echo "version=1.4.${{ github.run_number }}" >> "$GITHUB_OUTPUT"
    - run: make build
    - uses: actions/upload-artifact@v4
      with:
        name: dist
        path: dist/

deploy:
  needs: build
  runs-on: ubuntu-latest
  steps:
    - uses: actions/download-artifact@v4
      with:
        name: dist
        path: dist/
    - name: Deploy
      env:
        DEPLOY_TOKEN: ${ secrets.DEPLOY_TOKEN }
      run: ./deploy.sh --version "${ needs.build.outputs.version }"
    - name: Report
      run: echo "Deployed ${ needs.build.outputs.version }" >> "$GITHUB_STEP_SUMMARY"
```

The same shape for a JavaScript project, where only the setup action and the commands change, which is the point:



```
node-version: 20
cache: npm
- run: npm ci
- run: npm run lint
- run: npm test -- --coverage
- run: npm run build
```

#### ► TRY IT: THE ONE THAT MATTERS

1. Take this pipeline into a project you work on, adapting the commands to your language.
2. Get it green. Then deliberately break a test and confirm the report artifact still uploads.
3. Push to a branch and confirm `build` and `deploy` are skipped, then merge to main and watch all three run.
4. Add the status badge below to your README.

**Expected result** – a working pipeline on real code, and the confidence that comes from having built rather than read it. An hour here beats a second pass over the page.

## Show it off: a status badge

README.md

Copy

```
![CI](https://github.com/OWNER/REPO/actions/workflows/ci.yml/badge.svg)
```

The badge reflects the latest run on the default branch and links to the workflow's history. Add `?branch=develop` to point it at a different branch.

#### ► TRY IT

1. Add the badge to your README with your own owner, repo, and workflow filename.
2. Break the build on purpose and refresh the README to watch it turn red.



## What you can now do, and what comes next

You can write a workflow from scratch, choose triggers precisely, run jobs in parallel and order them, use and version Marketplace actions, branch on context values, manage environment variables and secrets, cache dependencies, keep artifacts, move data between jobs, test a matrix of versions, guard against hung jobs, and debug a red run. That is a working practitioner's toolkit, enough to own CI on a real project.

| Can you...                                       |                                                          |
|--------------------------------------------------|----------------------------------------------------------|
| Name the exact folder workflows live in?         | <code>.github/workflows/</code>                          |
| Explain CI versus CD?                            | Prove it is safe · ship what passed                      |
| Say why <code>actions/checkout</code> is needed? | The runner starts empty                                  |
| Explain why jobs cannot share files?             | Separate machines                                        |
| Explain why <code>cd</code> does not persist?    | New shell per step                                       |
| Make a second run faster?                        | <code>cache:</code> on the setup action                  |
| Keep a report from a failed run?                 | <code>upload-artifact</code> + <code>if: always()</code> |
| Deploy only from <code>main</code> ?             | <code>if: github.ref == 'refs/heads/main'</code>         |
| Pass a string between jobs?                      | Job <code>outputs</code> + <code>needs</code>            |
| Stop a hung job?                                 | <code>timeout-minutes</code>                             |

**Mid-level takes every one of those topics further:** the full context and function reference, cache key design and its two failure modes, artifact retention and collisions, matrix `include` / `exclude` /dynamic generation, plus the machinery you have not met yet: service



**Senior then covers what you own when CI/CD is your responsibility:** the trust model and script injection, least-privilege permissions, OIDC instead of stored cloud keys, supply-chain controls, container builds, authoring your own actions, self-hosted runners, and running Actions as a platform across many repositories.

NEXT UP

Beginner · Interview Review



Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



## LEARN

Roadmaps

Student Guides

Courses

The MLOps Practitioner

Sessions



**MLOps MENA**  
COMMUNITY

العربية



**Join the Community** ↗



## COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

## WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)

[Zomra](#)



### **Brainsmingle space**

Free to join this week with code **FREECORETEAM**